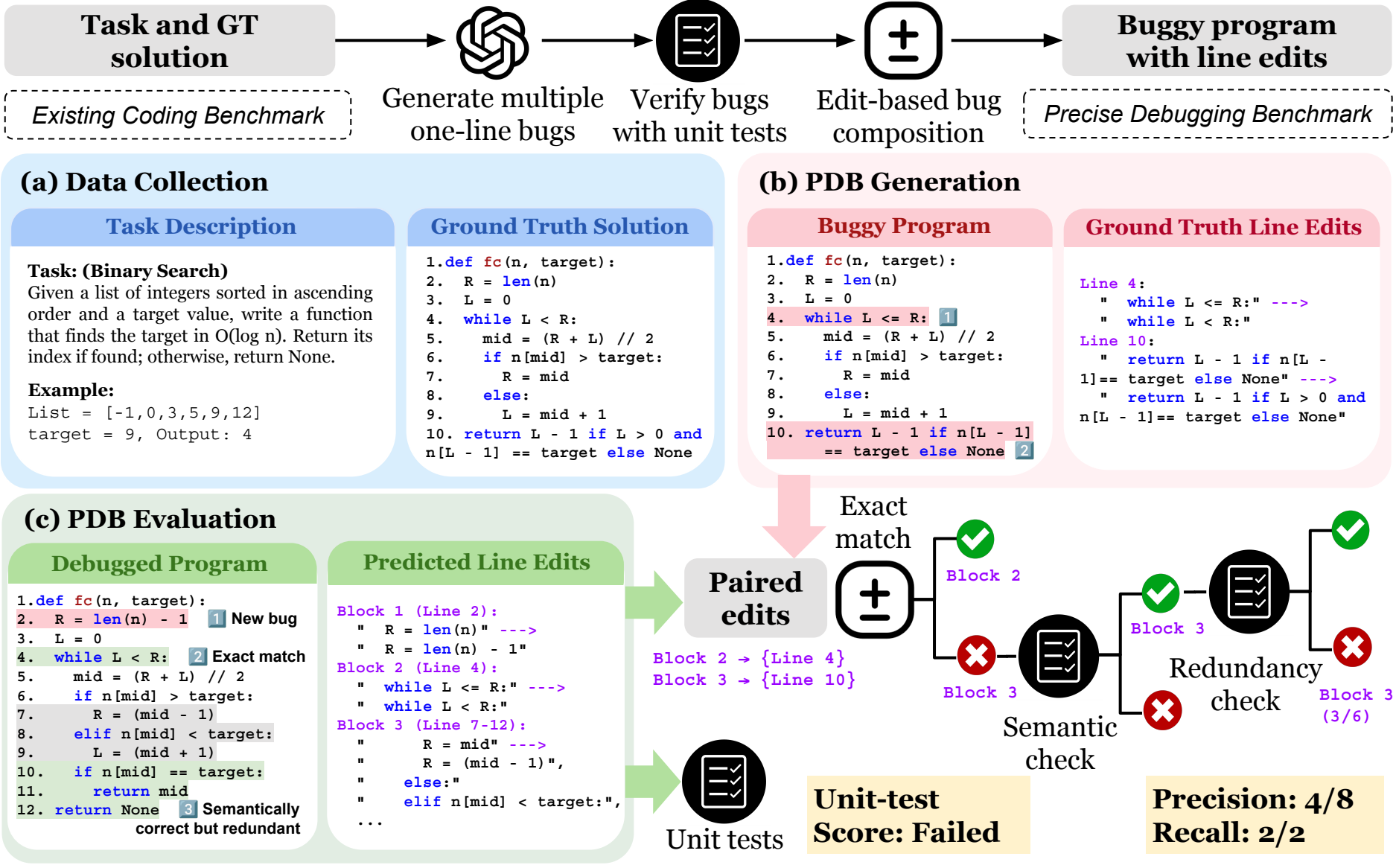
Task and GT solution
Existing Coding Benchmark
Generate multiple one-line bugs
Verify bugs with unit tests
Edit-based bug composition
Buggy program with line edits
Precise Debugging Benchmark
(a) Data Collection
Task Description
Task: (Binary Search)
Given a list of integers sorted in ascending order and a target value, write a function that finds the target in O(log n). Return its index if found; otherwise, return None.
Example:
List = [-1,0,3,5,9,12]
target = 9, Output: 4
Ground Truth Solution
1.def fc(n, target):
2. R = len(n)
3. L = 0
4. while L < R:
5. mid = (R + L) // 2
6. if n[mid] > target:
7. R = mid
8. else:
9. L = mid + 1
10. return L - 1 if L > 0 and n[L - 1] == target else None
(b) PDB Generation
Buggy Program
1.def fc(n, target):
2. R = len(n)
3. L = 0
4. while L <= R: 1
5. mid = (R + L) // 2
6. if n[mid] > target:
7. R = mid
8. else:
9. L = mid + 1
10. return L - 1 if n[L - 1] == target else None 2
Ground Truth Line Edits
Line 4:
" while L <= R:" --->
" while L < R:"
Line 10:
" return L - 1 if n[L - 1]== target else None" --->
" return L - 1 if L > 0 and n[L - 1]== target else None"
(c) PDB Evaluation
Debugged Program
1.def fc(n, target):
2. R = len(n) - 1 1 New bug
3. L = 0
4. while L < R: 2 Exact match
5. mid = (R + L) // 2
6. if n[mid] > target:
7. R = (mid - 1)
8. elif n[mid] < target:
9. L = (mid + 1)
10. if n[mid] == target:
11. return mid
12. return None 3 Semantically correct but redundant
Predicted Line Edits
Block 1 (Line 2):
" R = len(n)" --->
" R = len(n) - 1"
Block 2 (Line 4):
" while L <= R:" --->
" while L < R:"
Block 3 (Line 7-12):
" R = mid" --->
" R = (mid - 1)",
" else:"
" elif n[mid] < target:",
...
Paired edits
Block 2 → {Line 4}
Block 3 → {Line 10}
Exact match
Block 2
Block 3
Semantic check
Block 3
Redundancy check
Block 3 (3/6)
Unit tests
Unit-test Score: Failed
Precision: 4/8
Recall: 2/2